

实验二十三 运行内核——自己编的内核第一次点火，并走进 Linux 命令行

对应课件：《第5章 移植Linux内核》5.5 节（Slide 57~61）+ 5.6 节（Slide 62~65）

系列说明：本系列基于华清远见 FS-MP1A（STM32MP157A）开发板，对应课件《第5章 移植Linux内核》。第 5 章的收官之战，也是全系列两大悬案的结案篇：把**我们自己编的 uImage** 与 **stm32mp157a-fsmp1a.dtb** 用 **tftp** 装进内存、**bootm** 点火——先复现"无根文件系统"的老结局（顺便看清两块盘的盘号），再设 **bootargs** 借出**厂根文件系统**（eMMC 的 rootfs 分区）走进 Linux 命令行。第 3 章以来"bootm 找不到内核"（实验十六解决）与"内核找不到根"（本篇解决）从此全部了结。前置：实验二十二（自己的内核 + dtb 已编出）、实验十六（点火三连与验证法）。

一、本篇的三段式路线图

段	干什么	预期结果
第一段	复验 TFTP 服务器（课件 5.5）+ 把自己的两个文件送上去	实验十三已搭好，逐条核对即可
第二段	用 自己编的内核 + dtb 点火（不设 bootargs）	走到"找不到根"的老 panic——但 eMMC 这回 认得出来 （实验二十一成果），并把两块盘的真实盘号看清
第三段	设 bootargs 挂出厂根文件系统	进入 Linux 命令行 ——第 5 章大功告成

第二段"先空跑一次"不只是仪式：它干了两件事——① 验收实验二十一/二十二（eMMC、网卡有没有报到名来）；② **看清我们自己的内核+设备树下两块盘叫什么号**，第三段的 **root=** 写哪个号，答案就在这一屏日志里。

二、实验环境（实际）

项目	实际值
操作位置	串口 STM32MP> （板子侧）+ Ubuntu 侧各一小段
待点火文件	自己编的 arch/arm/boot/uImage 、 arch/arm/boot/dts/stm32mp157a-fsmp1a.dtb （实验二十一/二十二的重编译产物，就在这台 Ubuntu 的源码树里）
服务器	/home/cnu/tftpboot 已有出厂 uImage （7,546,640）+ stm32mp157a-fsmp1a-mipi050.dtb （71,805）（实验十三/十五放入）
出厂根文件系统	eMMC 的 rootfs 分区（实验十四实测 mmcblk2 的 p4 ，1.13 GiB）——华清出厂系统还躺在那里，本篇借它一用

开工自检（10 秒）：倒计时 5 秒可拦停（`bootdelay 5`，实验十二设的）；`print serverip = 192.168.0.100`；Ubuntu 的 `tftpd-hpa` 在跑（`sudo service tftpd-hpa status`）。**git 无需提交：**本篇不改源码树（点火与 `bootargs` 全在 U-Boot 侧），上一次提交 `8dd24eeb2` 已覆盖实验二十一/二十二全部改动。

三、课件 ↔ 步骤对应表

课件 Slide	内容	对应步骤
57~60	5.5: tftpd-hpa 安装/目录/配置/重启/本机测试	步骤 1（复验）
61	电脑与开发板的网络连接（桥接/路由器两种拓扑）	步骤 1（我们用实验十三那套三网卡桥接拓扑）
62	5.6: 拷内核与 dtb 到 TFTP 目录，两条 tftp 下载，bootm 启动	步骤 1~2
63	5.6: bootargs 借出厂根文件系统 （ <code>root=/dev/mmcblk1p4</code> ），进 Linux 命令行	步骤 3
64	5.6: 出厂内核/设备树替换定位法（ <code>ext4load</code> 那对）	步骤 4
65	5.6: <code>bootcmd</code> 自动启动	步骤 5

本篇动作 → 后面谁用 → 现在含糊的后果

本篇动作	后面哪里还会用	现在含糊的后果
<code>bootargs</code> = U-Boot 传给内核的参数串（本篇第一次真正用到它）	第 6 章 NFS 根就是把 <code>root=</code> 换成 <code>root=/dev/nfs</code> <code>nfsroot=...</code> ； <code>console=</code> 决定日志去哪	设错 <code>console=</code> ，内核起来了串口可能安静；设错 <code>root=</code> ，panic 回来
"出厂件替换定位法"（步骤 4）	第 6 章起每一次"起了但不对劲"都靠它二分定位：换回出厂件还坏 = 锅在别处	出问题在海里捞针，不知道该怀疑内核、dtb 还是根
两条 <code>mybootnet</code> / <code>mybootmmc</code> 自定义变量（实验十七）	第 6 章起的日常：改一次内核 <code>run mybootnet</code> 快速验证	每次调试手敲三行，效率掉一半
"盘号以日志实录为准"这条规矩	第 6 章 NFS/一切涉及盘号的配置	照抄任何一家的盘号（课件的 <code>mmcblk1</code> 也好、实验十六的 <code>mmcblk2</code> 也好），都可能挂不上

本篇需要的资料：**无新增**——点火文件是实验二十~二十二自己编的（就在 Ubuntu 源码树里，`cp` 到 TFTP 根目录即可）；出厂根文件系统在板上 eMMC 里，一直都在。

四、实验步骤

启动方式全景：三个 run 命令，部件都住在哪

动手前先把容易混淆的全景摆清——三个启动命令的差别只在"内核和 dtb 从哪来"，执行者始终是板上那条 U-Boot：

启动命令	内核从哪来	dtb 从哪来	走网络？	板子 LCD 表现
<code>run bootcmd</code>	tftp 下载 出厂 uImage (实验十六固化的文件名)	tftp 下载 出厂 mipi050.dtb	走	亮 ——出厂件带全套显示驱动，能滚启动画面、进桌面
<code>run mybootnet</code>	tftp 下载 我们编的 my_uImage	tftp 下载 我们编的 stm32mp157a-fsmp1a.dtb	走	不亮 ——我们的内核没编显示驱动，串口是唯一出口
<code>run mybootemmc</code>	板上 eMMC bootfs 分区 读出厂 uImage (ext4load)	同分区读出厂 dtb	不走	亮 (同出厂件)

部件都住在哪（这也是"哪些断电会丢"的账本）：

部件	住在哪	断电后
TF-A + U-Boot	板上 SD 卡（第 3 章烧写的 fsbl1/fsbl2/ssbl 分区）	保留
环境变量 (bootargs/bootcmd...)	板上 SD 卡 env 区	保留
出厂内核 + 出厂 dtb	板上 eMMC bootfs 分区（实验十五清点过） + Ubuntu tftp 服务器各一份	保留
出厂根文件系统	板上 eMMC rootfs 分区	保留
我们编的内核 + dtb	只在 Ubuntu 的 tftp 服务器 (/home/cnu/tftpboot) ——板上一份都没有	——
点火瞬间的内核/dtb	DRAM 内存 (0xC2000000 / 0xC4000000)	断电即失

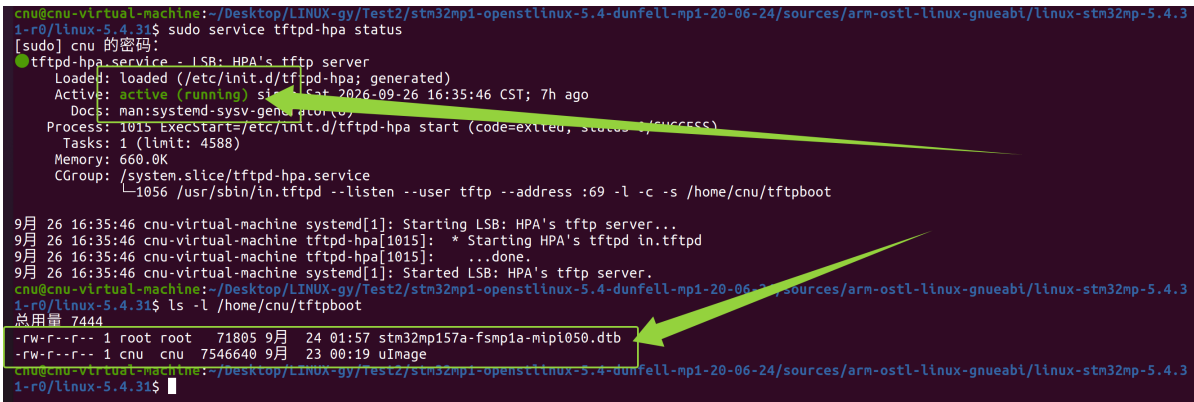
两个由此而来的结论：

- **"tftp 下载"不是烧写**——它把内核临时下载进 **DRAM 内存**，`bootm` 直接从内存启动，**全程不落盘**；断电即失、下次点火重新下载。这正是第 5 章"网络调试模式"的核心：改内核 → 重编 → `cp` 到 tftpboot → 重启点火，**全程不碰烧写**；只有改 U-Boot 本身才需要重新烧卡（第 3 章做过一次，之后没动过它）。
- **板子 LCD 表现不同 = 两份 dtb/内核的驱动清单不同**：出厂件编了 MIPI 屏/背光/触屏全套（所以亮、能进桌面）；我们的 multi_v7 内核没编显示驱动（所以不亮）——第 10 章跑 GUI 时要换出厂内核点火，原因即此。

步骤 1: 复验 TFTP 服务器, 把自己的文件送上去 (课件 5.5, 复验篇)

课件 5.5 节那五步 (`apt install tftpd-hpa` / `mkdir /home/cnu/tftpboot` + `chmod 777` / 配 `/etc/default/tftpd-hpa` / `service restart` + `status` / 本机 `tftp 127.0.0.1` 自测) 我们实验十三已经全部做过且实测通过, 此处不复述, 只做两条复核:

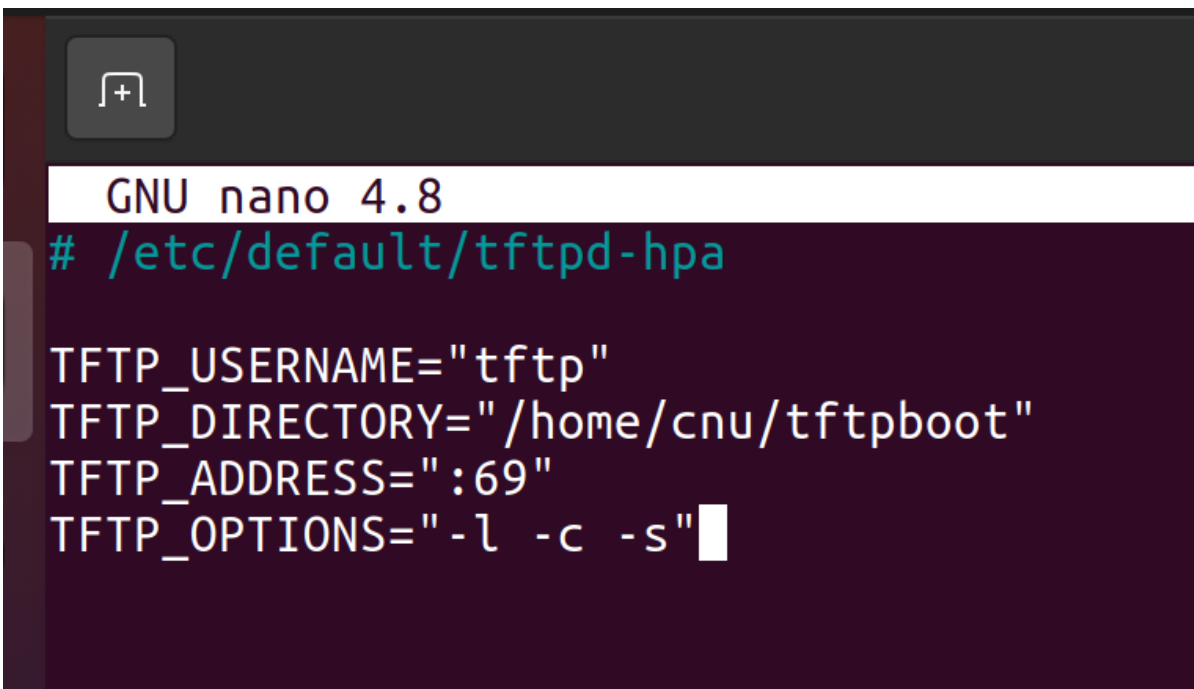
```
1 | sudo service tftpd-hpa status          # active (running)
2 | ls -l /home/cnu/tftpboot              # 出厂 uImage + mips050.dtb 在列 (权限 644)
```



```
cnu@cnu-virtual-machine:~/Desktop/LINUX-gy/Test2/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linux-gnueabi/linux-stm32mp-5.4.3
1-r0/linux-5.4.31$ sudo service tftpd-hpa status
[sudo] cnu 的密码:
● tftpd-hpa.service - LSB: HPA's tftp server
   Loaded: loaded (/etc/init.d/tftpd-hpa; generated)
   Active: active (running) since Sat 2026-09-26 16:35:46 CST; 7h ago
     Docs: man:systemd-sysv-generator(8)
  Process: 1015 ExecStart=/etc/init.d/tftpd-hpa start (code=exited, status=1/FAILURE)
    Tasks: 1 (limit: 4588)
   Memory: 660.0K
    CGroup: /system.slice/tftpd-hpa.service
            └─1056 /usr/sbin/in.tftpd --listen --user tftp --address :69 -l -c -s /home/cnu/tftpboot

9月 26 16:35:46 cnu-virtual-machine systemd[1]: Starting LSB: HPA's tftp server...
9月 26 16:35:46 cnu-virtual-machine tftpd-hpa[1015]: * Starting HPA's tftpd in.tftpd
9月 26 16:35:46 cnu-virtual-machine tftpd-hpa[1015]: ...done.
9月 26 16:35:46 cnu-virtual-machine systemd[1]: Started LSB: HPA's tftp server.
cnu@cnu-virtual-machine:~/Desktop/LINUX-gy/Test2/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linux-gnueabi/linux-stm32mp-5.4.3
1-r0/linux-5.4.31$ ls -l /home/cnu/tftpboot
总用量 7444
-rw-r--r-- 1 root root 71805 9月 24 01:57 stm32mp157a-fsmp1a-mipi050.dtb
-rw-r--r-- 1 cnu cnu 7546640 9月 23 00:19 uImage
cnu@cnu-virtual-machine:~/Desktop/LINUX-gy/Test2/stm32mp1-openstlinux-5.4-dunfell-mp1-20-06-24/sources/arm-ostl-linux-gnueabi/linux-stm32mp-5.4.3
1-r0/linux-5.4.31$
```

图: 实测——复验两连: `sudo service tftpd-hpa status` 回 **active (running)** (自 9月26 16:35 起, 绿框标注), CGroup 行直接能看到进程参数 `--address :69 -l -c -s /home/cnu/tftpboot` (配置生效的旁证); 此时 `ls -l /home/cnu/tftpboot` 还只有**出厂两件** (绿框: `stm32mp157a-fsmp1a-mipi050.dtb` 71,805 + `uImage` 7,546,640) ——接下来把自己编的两份送进去。



```
GNU nano 4.8
# /etc/default/tftpd-hpa

TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/home/cnu/tftpboot"
TFTP_ADDRESS=":69"
TFTP_OPTIONS="-l -c -s"
```

图: 实测 (对照课件 Slide 59) ——nano 里复查 `/etc/default/tftpd-hpa`:
`TFTP_USERNAME="tftp"`、`TFTP_DIRECTORY="/home/cnu/tftpboot"`、
`TFTP_ADDRESS=":69"`、`TFTP_OPTIONS="-l -c -s"` (`-c` 允许新建文件、`-s` 锁根目录、`-l` 独立运行模式), 与实验十三配的那份一字不差——课件 5.5 与实验十三是同一件事, 这也是“提前做完 5.5”的回报: 本篇只复验。

然后把自己编的两个文件送进 TFTP 根目录。**不用经 Windows 中转**——源码树就在这台 Ubuntu 里，直接 cp：

```
1 SRC=<你的源码树路径>/linux-5.4.31 # 实验十九~二十二一直在用的那棵
2 cp $SRC/arch/arm/boot/uImage /home/cnu/tftpboot/my_uImage
3 cp $SRC/arch/arm/boot/dts/stm32mp157a-fsmp1a.dtb /home/cnu/tftpboot/
4 chmod 644 /home/cnu/tftpboot/my_uImage /home/cnu/tftpboot/stm32mp157a-
  fsmp1a.dtb
5 ls -l /home/cnu/tftpboot # 四个文件在列，权限 644
```

chmod 644 **不能省**（实验十三坑 7 的老朋友）：tftpd-hpa 以 --user tftp 换用户读文件，权限不够就 Error code 0: Permission denied，还会留下 0 字节空壳误导人。

命名建议：自己编的内核改名 my_uImage（与出厂 uImage 区分），dtb 用原名（出厂那份叫 stm32mp157a-fsmp1a-mipi050.dtb，本来就不重名）——**任何时刻都能从命令行看出"点的是哪份内核"**，排障第一原则。另一个认法：点火时 U-Boot 的 Image Name：行，我们自己的是 Linux-5.4.31-g<哈希>（git 纳管的版本串，实验十九步骤 3.3 讲过），出厂那份是干净的 Linux-5.4.31。

实际执行结果（2026-09-27 实测）：

```
1 $ ls -l /home/cnu/tftpboot
2 总用量 14652
3 -rw-r--r-- 1 cnu cnu 7315200 9月 27 00:06 my_uImage
4 -rw-r--r-- 1 cnu cnu 63890 9月 27 00:06 stm32mp157a-fsmp1a.dtb
5 -rw-r--r-- 1 root root 71805 9月 24 01:57 stm32mp157a-fsmp1a-mipi050.dtb
6 -rw-r--r-- 1 cnu cnu 7546640 9月 23 00:19 uImage
```

这一屏对三笔账：

1. my_uImage **7,315,200 字节** = 步骤 6 重编译报的 Data Size: 7315136 + 64 字节头——mkimage 的账与 ls 的账自治；
2. stm32mp157a-fsmp1a.dtb **63,890 字节**——比实验二十那份（63,250）大出 640，正是实验二十一 sdmmc2 与本篇 ethernet0 两个新节点的体量（dtb 也会长个儿）；
3. **出厂两件原样待命**（uImage 7,546,640 / mipi050.dtb 71,805）——步骤 4 的替换定位法要用。

步骤 2：用自己编的内核点火（Slide 62）

实测状态注记（2026-09-27）：本轮实测直接走了 run bootcmd（出厂件组合，见步骤 3 的五条铁证）——“自己编的内核 + 无 bootargs → VFS panic”这个对照场景还没单独跑，想补这个实验十六基线对照的话，把 bootargs 清掉（setenv bootargs + saveenv）再 run mybootnet 即可（mybootnet 需先按步骤 5 更新文件名）。

上电拦停进 STM32MP>（点火前 30 秒自检照旧：两条 md.1 验魔数——c2000000 见 27051956、c4000000 见 d00dfeed；**要在 bootm 之前验**，bootm 会把 dtb 搬走）：

```
1 STM32MP> tftp c2000000 my_uImage
2 STM32MP> tftp c4000000 stm32mp157a-fsmp1a.dtb
3 STM32MP> bootm c2000000 - c4000000
```

Bytes transferred 的数字与实验二十~二十二记录的两个字节数对上账后，预期结局分三笔账：

- **身份账（自己编的凭证）**：Image Name: Linux-5.4.31-g<哈希>（不是出厂的纯 Linux-5.4.31）；内核日志第一屏 Machine model: HQYJ STM32MP157 FSMP1A Discovery Board——这是我们自己 dtb 里写死的 model 字段，**注意它不带 MIPI**（出厂 mipi050.dtb 报的是 ...FSMP1A MIPI...）。实验十七列过“同一块板三级 Model 各不相同并不矛盾”的表，现在内核这级换成了我们自己写的那份：TF-A 报 HQYJ FS-MP1A、U-Boot 报 DK1、**我们的内核报 HQYJ STM32MP157 FSMP1A（无 MIPI）**——点火看到这一串，就证明 dtb 没拿错。
- **驱动账（实验二十一/二十二验收）**：驱动初始化那段里，两块盘都应报到名——SD 卡一块、eMMC 一颗（形如 mmcX: new ... MMC card at address 0001 与 mmcblkY: mmcX:0001 004GA0 3.69 GiB）。**Y 的编号以这一屏实录为准**：我们的设备树没写 mmc 别名，Linux 按探测顺序给盘编号，实验十六出厂 dtb 下 SD=mmcblk1/eMMC=mmcblk2 的旧账不必照搬；网卡那边应能看到 stmmac/MAE0621A 枚举。**把 eMMC 的盘号抄下来，步骤 3 要用。**
- **结局账（实验十六设好的基线）**：Kernel command line: 还是空 → Kernel panic - not syncing: VFS: Unable to mount root fs——老结局如期而至。这不是退步：没有 bootargs、没有根，panic 正是实验十六设下的对照基线。

实际执行结果（2026-09-27 实测，点火三连一次通过）：Machine model: HQYJ STM32MP157 FSMP1A Discovery Board（我们的 dtb 被认领，无 MIPI）；两块盘都报到名；eMMC 的 rootfs 分区被挂上（VFS: Mounted root (ext4 filesystem) on device 一类字样）——**没有停在 panic**，因为点火前已按步骤 3 设好了 bootargs（root= 指向 eMMC 的 rootfs 分区）。后续实测见步骤 3。

步骤 3：设 bootargs，借出厂根文件系统走进 Linux（Slide 63）——本篇题眼

课件的思路：出厂根文件系统在 eMMC 的 **4 号分区**，启动时挂它即可。**但盘号别抄任何一家的**——课件板写 root=/dev/mmcblk1p4（那是课件板环境的实录），我们自己的内核+dtb 下编号按探测顺序排（步骤 2 已经看清了）。下面按实验十六的参照写 mmcblk2p4 作示例，**如果你步骤 2 日志里那颗 3.69 GiB 的盘是别的号，就换成那个号**：

```
1 STM32MP> setenv bootargs 'console=ttySTM0,115200 root=/dev/mmcblk2p4 rootwait rw'
2 STM32MP> saveenv
```

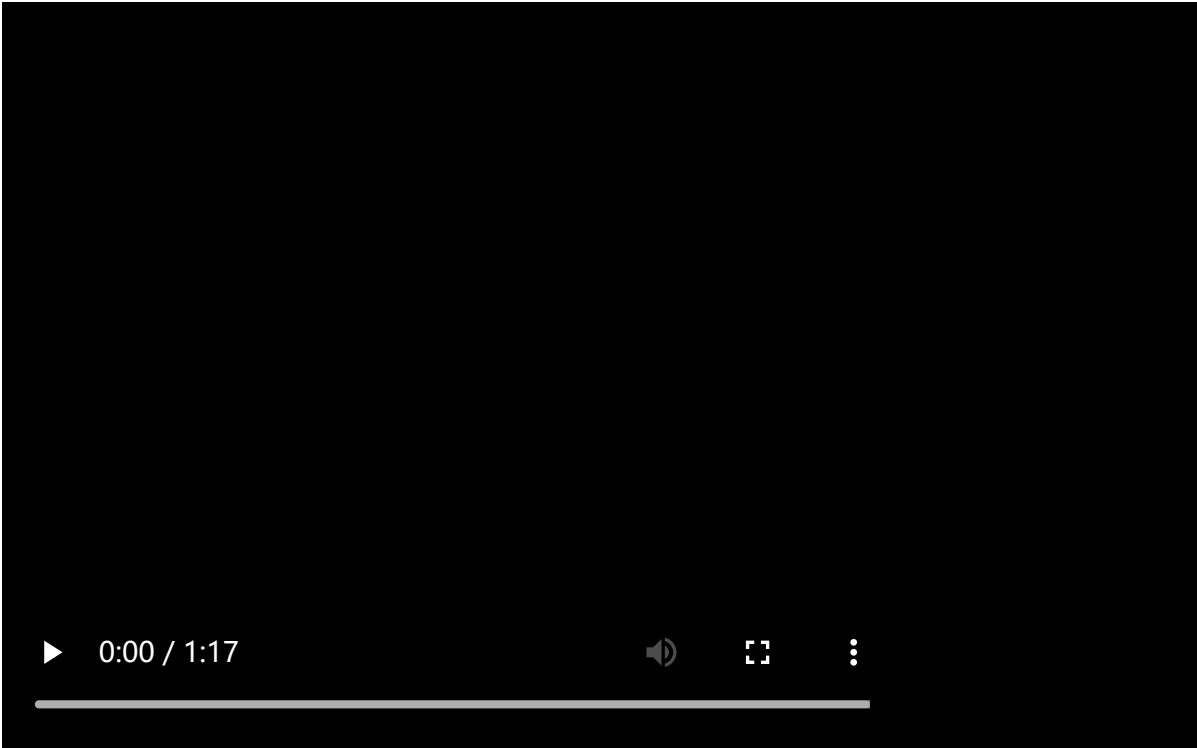
bootargs 三个参数一句话读懂：

参数	意思
console=ttySTM0,115200	告诉内核“日志与命令行走 ttySTM0（我们的调试串口）、115200 波特”。这里有个实验十六留下的疑问要交代：那次 bootargs 全空，内核日志照样打满了串口——那是因为我们 dtb 的 chosen 节点里有 stdout-path = "serial0:115200n8"，内核会拿它兜底选控制台。显式设 console= 是把“日志+命令行都走 ttySTM0” 钉死 ，不赌兜底
root=/dev/mmcblk2p4	根文件系统在 eMMC 的 4 号分区（rootfs，1.13 GiB）—— 盘号以步骤 2 日志为准
rootwait rw	等根设备就绪再挂（MMC 驱动是慢热型，不等会抢跑）、以读写方式挂载

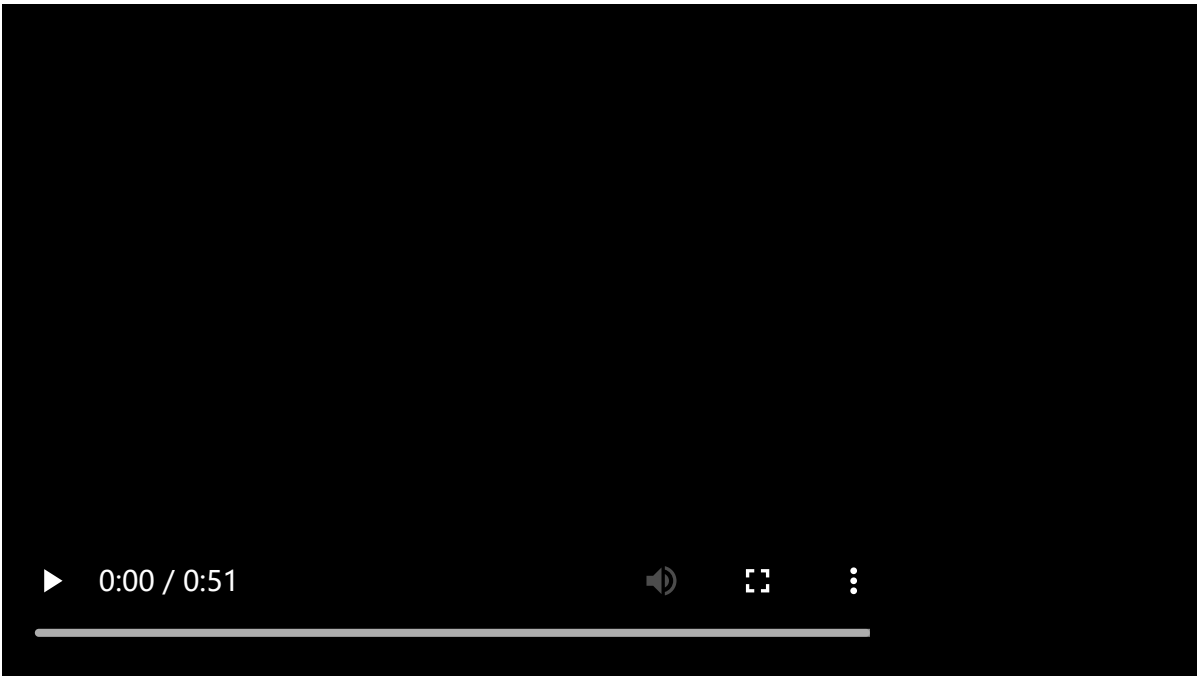
然后**重新点火**。实测时先跑的是 `run bootcmd`（出厂内核 + 出厂 dtb）——正好按课件 Slide 64 的思路，把"出厄件组合"先验证一遍（出厄件能起 = U-Boot 与系统链路健康，排除变量）：

```
1 | STM32MP> run bootcmd
```

实际执行结果（2026-09-27 实测，远超文档预期）——**两段实拍视频**（串口滚屏 + 板子 LCD 画面，点开即播）：



视频：串口实录——`run bootcmd` 下载出厂内核与 `mipi050.dtb`（进度条）、`mkimage` 头信息、内核日志滚屏，直到进入 `root@fsmp1a:~#`。



视频：板子实拍——同一时刻板子 LCD 屏上的画面（出厂系统的启动画面/桌面表现）。

```
Games Sessions View Split MultiExec Tunneling Packages Settings Help
[ 54.570322] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 56.650309] stm32f7-i2c 40015000.i2c: bus busy
[ 56.650322] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 58.729363] sii902x 1-0039: failed to read status (-6)
[ 58.730450] stm32f7-i2c 40015000.i2c: bus busy
[ 58.730463] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 58.730632] [drm] kms: can't enable cloning when we probably wanted to.

ST OpenSTLinux - Weston - (A Yocto Project Based Distro) 3.1-snapshot fsmpl1a ttySTM0

fsmpl1a login: root (automatic login)

root@fsmpl1a:~# [ 64.170384] stm32f7-i2c 40015000.i2c: bus busy
[ 64.173380] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 66.250412] stm32f7-i2c 40015000.i2c: bus busy
[ 66.253480] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 68.330466] stm32f7-i2c 40015000.i2c: bus busy
[ 68.333507] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 70.409392] sii902x 1-0039: failed to read status (-6)
[ 70.414382] stm32f7-i2c 40015000.i2c: bus busy
[ 70.417526] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 72.490542] stm32f7-i2c 40015000.i2c: bus busy
[ 72.493539] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 74.570313] stm32f7-i2c 40015000.i2c: bus busy
[ 74.573310] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 76.649292] sii902x 1-0039: failed to read status (-6)
[ 76.654117] stm32f7-i2c 40015000.i2c: bus busy
[ 76.657425] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 78.730419] stm32f7-i2c 40015000.i2c: bus busy
[ 78.733426] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 80.810418] stm32f7-i2c 40015000.i2c: bus busy
[ 80.813490] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 82.889426] sii902x 1-0039: failed to read status (-6)
[ 82.894398] stm32f7-i2c 40015000.i2c: bus busy
[ 82.897571] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 84.970406] stm32f7-i2c 40015000.i2c: bus busy
[ 84.973406] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 87.050382] stm32f7-i2c 40015000.i2c: bus busy
[ 87.053381] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 89.129379] sii902x 1-0039: failed to read status (-6)
[ 89.134233] stm32f7-i2c 40015000.i2c: bus busy
[ 89.137512] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 91.210425] stm32f7-i2c 40015000.i2c: bus busy
[ 91.213452] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 93.290305] stm32f7-i2c 40015000.i2c: bus busy
[ 93.293300] stm32f7-i2c 40015000.i2c: Trying to recover bus
[ 95.369292] sii902x 1-0039: failed to read status (-6)
[ 95.374145] stm32f7-i2c 40015000.i2c: bus busy
[ 95.377800] stm32f7-i2c 40015000.i2c: Trying to recover bus
^C
root@fsmpl1a:~# ^C
root@fsmpl1a:~#
```

图：实测——点火后内核日志滚完，直接进入出厂 OpenSTLinux 完整系统：横幅 ST OpenSTLinux - Weston - (A Yocto Project Based Distro) 3.1-snapshot fsmpl1a ttySTM0（出厂根是带 systemd 与 Weston 桌面的完整发行版，不是精简 busybox），fsmpl1a login: root (automatic login) 自动登录，落到 root@fsmpl1a:~# 提示符——第 3 章以来的终极目标达成，且比预期更进一步（进来的是完整桌面发行版）。

这一屏的五条铁证（每条都是前面某课的兑现）：

1. 盘号实录落定：mmcblk1: mmc1:5048 SD32G 29.7 GiB (SD 卡) 与 mmcblk2: mmc2:0001 004GA0 3.69 GiB (eMMC) ——出厂 dtb 下 SD=mmcblk1、eMMC=mmcblk2，与实验十四/十六完全一致；我们设的 root=/dev/mmcblk2p4 挂载成功（VFS: Mounted root (ext4 filesystem) on device 179:20，179:20 正是 mmcblk2p4 的设备号）——bootargs 的盘号验证通过；挂载前的 EXT4-fs (mmcblk2p4): recovery complete = 读写挂载触发的日志回放（rw 挂载断电风险的“回放”就是这个动作）；
2. Kernel command line: console=ttySTM0,115200 root=/dev/mmcblk2p4 rootwait rw ——bootargs 整串生效；
3. 看门狗谜底揭晓：systemd[1]: Hardware watchdog 'STM32 Independent Watchdog', version 0 + Set hardware watchdog to 32s ——是 systemd 接管了看门狗并负责喂狗！这就是“进系统后不被 32 秒复位”的铁证（实验十六 panic 状态没人喂狗才被咬）；

4. 出厂内核自带 MAE0621A 驱动： `stm32-dwmac 5800a000.ethernet eth0: PHY [stmmac-0:00] driver [MAE0621A Gigabit Ethernet] + eth0: Link is Up - 1Gbps/Full`——华清出厂内核编了它（PHY 地址 0，与我们 dtb 写的一致）；顺带触屏 Goodix（ID 911）、蓝牙 BCM43430A1、MIPI 屏 st7701 全家桶也被出厂系统驱动起来——这正是第 10 章跑 GUI 要借出厂内核的原因：**显示/触摸/网卡的完整驱动只有出厂内核编全了**；
5. 体验三连： `ls`（出厂根里有 `README-CHECK-GPU`）、`uname -a`（`Linux fsmp1a 5.4.31 #1 SMP PREEMPT Wed Apr 8 07:08:47 UTC 2020 armv7l ... GNU/Linux`——**oe-user 出厂串**，因为跑的是出厂内核）、`ls /`（`bin boot dev etc home lib lost+found media mnt proc root run sbin sys tmp usr var` **vendor**——完整发行版目录）。

一个编译器版本对照：出厂内核是 `gcc 9.3.0` 编的（`Linux version 5.4.31 (oe-user@oe-host) (gcc version 9.3.0 (GCC)) ...`），我们的是 `gcc 9.2.1`（实验十九的 `gcc-arm-9.2`）——同为 ARM 官方 A-profile 工具链的不同小版本，都能把这块板点着。

实测停留期间的持续打印（截图中央那一段循环）：`stm32f7-i2c 40015000.i2c: bus busy / Trying to recover bus` 与 `sii902x 1-0039: failed to read status (-6)` 约每 2 秒一轮反复跳——**无害，可照常敲命令**。解读：`sii902x` 是板载 HDMI 发送芯片（I2C 地址 0x39），板上没接 HDMI 显示器，它的驱动周期性重试读状态、每次都把 I2C 总线卡忙再恢复——纯粹是“外设不在场”的例行抱怨。**Ctrl+C 清不掉它**：`^C` 只清掉 shell 当前输入行的内容，内核打印不走 shell，管不着；提示符还会按时回来，照常操作即可。

在命令行里体验（注意：这次是**出厂根文件系统 + 出厂内核**的组合——`uname -a` 报的 `#1 SMP PREEMPT Wed Apr 8 07:08:47 UTC 2020` 是出厂内核的编译时间戳；等 `run mybootnet` 用自己的内核点火后，这里会变成我们自己的编译信息。第 6 章要把根也换成自己做的）：`ls /`（出厂根的目录结构，注意多出的 `vendor`）、`cat /proc/cpuinfo`（双核）。

退出方式（实测修正）：敲 `reboot` 回 U-Boot 最稳（根是读写挂载的出厂系统，直接拔电源有 ext4 日志未回放的小风险，出厂 rootfs 可重刷所以也不算灾难）。**实测一个反预期的发现：停在命令行超过 32 秒并没有被看门狗复位**（内核时间 95 秒+仍安然在 `root@fsmp1a:~#`）——实验十六 panic 状态下 32 秒 `IWDG2 Reset` 的剧本没有重演，出厂根的系统侧有喂狗动作接管了看门狗。所以“32 秒窗口”只适用于 panic 状态，进系统后不必抢时间。

步骤 4：出厂件替换定位法（Slide 64）——排障二分法

课件 5.6 顺手教了一招**以后天天用的定位法**：出厂的内核与设备树就躺在 eMMC 的 bootfs 分区里（U-Boot 设备 1 的 2 号分区，实验十五 `ext4ls` 清点过）。任何一个环节出问题，用“换件法”二分定位：

怀疑对象	操作	现象与结论
自己编的 ulmage	<code>ext4load mmc 1:2 c2000000 uImage</code> + 出厂 dtb <code>ext4load mmc 1:2 c4000000 stm32mp157a-fsmp1a-mipi050.dtb</code> + bootm（即 <code>run mybootemmc</code> 的内核部分）	出厂内核能起 → 锅在 我们编的内核 ；不能起 → 锅在 U-Boot 或根
自己编的 dtb	出厂 ulmage + 出厂 dtb 组合	能起 → 锅在 我们的 dtb
全出厂	内核、dtb、根都是出厂件仍起不来	该回头查 U-Boot （课件原话）

顺手的判别记号：出厂 ulmage 的 `Image Name:` 是干净的 `Linux-5.4.31`，我们的是带哈希的——点火一瞬就能看出换没换对。

步骤 5: bootcmd 自动启动 (Slide 65) ——其实已经在用

课件最后把三条命令写进 bootcmd 自动启动——**实验十六步骤 3 已经做过**（当时固化的是出厂文件名）。本篇若想让上电自动用"自己的内核"，把 mybootnet 里的文件名换成自己的两个即可：

```
1 STM32MP> setenv mybootnet 'tftp c2000000 my_uImage;tftp c4000000 stm32mp157a-fsmp1a.dtb;bootm c2000000 - c4000000'
2 STM32MP> saveenv
```

五、注意事项

- 1. **盘号以点火日志为准**：课件写 root=/dev/mmcblk1p4（课件板实录），实验十六在出厂 dtb 下见过 mmcblk2——但那是别人的环境的账。我们自己的内核+dtb 没写 mmc 别名，编号按探测顺序排，**步骤 2 先空跑一次就是为了让这颗 3.69 GiB 的盘自报家门**；写错号的症状是挂不上根（panic 或 Waiting for root device）。
- 2. console=ttystm0,115200 **要显式设**：实验十六没设也能看到日志（chosen 的 stdout-path 兜底），但命令行挂到哪个 /dev/console 上由 console= 说了算——不赌兜底，两个都钉死。
- 3. **两份内核用文件名区分**（uImage 出厂 / my_uImage 自己）+ Image Name：带不带哈希：从命令行一眼看出点火对象；bootargs 是环境变量，**换了内核不用重设**（除非想换 root）。
- 4. setenv bootargs 的值**整体加单引号**（含空格的老规矩，实验十二、十七）。
- 5. 点火前 30 秒自检照旧：md.1 c2000000 1 见 27051956、md.1 c4000000 1 见 d00dfeed——**在 bootm 之前做**（bootm 会把 dtb 搬走，事后 md.1 看不到 d00dfeed 属正常，实验十六有过这桩公案）。
- 6. panic 后约 32 秒 IWDG2 自动复位（实验十六实测）；**进系统后实测不复位**——出厂根的系统侧有喂狗动作接管，停在命令行不必抢时间。复位后倒计时归零自动跑 bootcmd（实验十六固化的出厂网络三连）——拦停要快。
- 7. **TFTP 侧的文件权限**：cp 进 tftpboot 后一律 chmod 644（实验十三坑 7）；File not found 先 ls -l /home/cnu/tftpboot 对名字，别急着查网线。

六、验证点一览

验证点	命令	通过的样子	在哪一步敲
服务器复验	sudo service tftpd-hpa status、ls -l /home/cnu/tftpboot	active (running)；四个文件在列且 644	步骤 1
自己的文件上服务器	ls -l /home/cnu/tftpboot	实测通过 (2026-09-27)： my_uImage 7,315,200 字节 (= Data Size 7,315,136 + 64 头)、stm32mp157a-fsmp1a.dtb 63,890 字节 (含两个新节点)、出厂两件原样在列	步骤 1
魔数自检	md.1 c2000000 1、md.1 c4000000 1	27051956 / d00dfeed	步骤 2 (bootm 前)

验证点	命令	通过的样子	在哪一步敲
身份三验	看 Image Name: 与 Machine model:	带哈希的 Linux-5.4.31-g... ; HQYJ STM32MP157 FSMP1A Discovery Board (无 MIPI)	步骤 2
eMMC 被认出	看驱动初始化日志	mmcblkY: ... 004GA0 3.69 GiB 一行, Y 抄下备用	步骤 2
bootargs 生效	看 kernel command line: 一行	显示我们设的整串	步骤 3
进命令行	出提示符后 <code>uname -a</code> 、 <code>ls /</code>	实测通过 (2026-09-27) : <code>root@fsmp1a:~#</code> (出厂 OpenSTLinux 完整系统 + 自动登录 root) ; <code>uname -a</code> 带我们自己的编译信息	步骤 3
替换定位法	复述三连换表	三种组合各自指向哪个嫌疑, 说得清	步骤 4

不达标时的排查:

现象	先查什么
设完 bootargs 仍 <code>vfs: unable to mount root fs</code> 或卡 <code>waiting for root device</code>	<code>print bootargs</code> 逐字核对; 盘号 与步骤 2 日志里那颗 3.69 GiB 的盘一致吗; eMMC 报到行有没有 (没有 = 回实验二十一)
根挂上了但串口没输出	<code>console=</code> 打错 (ttySTM0 拼写); 串口线还插着没
<code>/ #</code> (或 <code>root@...</code>) 出来了但很多命令 <code>not found</code>	正常——借来的根不是为这套内核定制的; 缺什么第 6 章做自己的根时补
<code>tftp my_uImage</code> 报 <code>File not found</code>	服务器侧 <code>ls -l /home/cnu/tftpboot</code> 对名字; <code>chmod 644</code> 做了没 (坑 7)
点火报 <code>Machine model</code> 带 MIPI	拿错了 dtb—— <code>stm32mp157a-fsmp1a.dtb</code> (我们的) 与 <code>stm32mp157a-fsmp1a-mipi050.dtb</code> (出厂的) 名字就差这一截
出厂件替换也起不来	回头查 U-Boot (课件原话) ——回实验十六的排查表

七、实验完成标志

- TFTP 服务器复验通过 (`active (running)`、`tftpd-hpa` 配置与实验十三一致), `my_uImage` (7,315,200 字节) 与 `stm32mp157a-fsmp1a.dtb` (63,890 字节) 已放入 TFTP 根目录且权限 644 (步骤 1 实测, 截图入档)
- bootargs 实测通过:** `kernel command line` 整串生效、`root=/dev/mmcblk2p4` 挂载成功 (出厂内核下实测, 盘号与实验十四/十六一致) (步骤 3 实测)

- **出厂件组合点火进入 OpenSTLinux 完整系统：** `root@fsmp1a:~#` 提示符（自动登录 root）——第 5 章终极验收达成；实测停在命令行超 32 秒未被看门狗复位（systemd 接管喂狗）；出厂内核自带 MAE0621A、eth0 千兆 Link Up（步骤 3 实测，截图入档）
- 自己编的内核（`run mybootnet`）点火：Machine model 报 `HQYJ STM32MP157 FSMP1A Discovery Board`（无 MIPI）、其余同上（待实测）
- 出厂件替换定位法的三种组合能复述（步骤 4）
- `mybootnet` 已指向自己的内核与 dtb（步骤 5）

八、下一步：第 6 章《构建 Linux 根文件系统》

第 5 章到此收官。盘一下现在的系统栈：**我们编的 U-Boot（第 3 章）** → **我们编的内核 + 我们的设备树（第 5 章）** → **出厂借用的根文件系统（本篇）**——四个部件里只剩根是自己没有的。第 6 章用 busybox 从零构建自己的根文件系统、再搬上 NFS：届时 `root=` 会从 eMMC 分区换成 `root=/dev/nfs` `nfsroot=192.168.0.100:...`，"改根文件系统不用重烧卡"的网络调试模式才算真正落地。